Aplicação de Ferramentas de Automação (Netmiko, NAPALM e Jinja2) na Configuração de Roteadores Cisco em Ambiente Simulado de Provedor de Serviços de Internet

Gabriel Henrique de Salles

Escola Politécnica – Pontifícia Universidade Católica do Paraná (PUCPR)

salles.gabriel@pucpr.edu.br

Resumo. *Este trabalho aplica Netmiko, NAPALM e Jinja2 à ativação de clientes corporativos em roteadores Cisco de um provedor de Internet, reproduzido no Cisco Modeling Labs com core, bordas, OSPF e iBGP com Route Reflector. O fluxo valida os dados, verifica a rede antes e depois da mudança e desfaz a configuração em caso de falha. O tempo médio de ativação caiu de 162 s para 5,7 s (−96,5%), sem os cinco erros do processo manual; os sete erros injetados foram barrados e o tempo cresceu de forma linear com o número de clientes. A auditoria automatizada detectou nove de nove desvios em 4 s, contra sete de nove em 346 s na manual. Na falha de uma borda, os clientes com acesso redundante foram ativados na contingência, sem configuração parcial nos demais.*

Abstract. *This work applies Netmiko, NAPALM and Jinja2 to the activation of business customers on Cisco routers of an Internet service provider, reproduced in Cisco Modeling Labs with core and edge routers, OSPF and iBGP with a Route Reflector. The workflow validates input data, checks the network before and after each change and rolls it back on failure. Mean time dropped from 162 s to 5.7 s (−96.5%), without the five manual errors; all seven injected errors were blocked and time grew linearly with the number of customers. The automated audit detected nine of nine deviations in 4 s, against seven of nine in 346 s manually. When an edge failed, redundant customers were activated on the backup edge, with no partial configuration.*

1. Introdução

Provedores de serviços de Internet (ISPs) operam redes com centenas ou milhares de roteadores, organizadas em camadas de *core* e de borda. Na borda, uma das tarefas mais frequentes é a ativação de clientes corporativos: configurar o ponto de entrega do serviço, identificar o circuito e anunciar o prefixo do cliente no BGP. Em muitos

provedores, essa tarefa ainda é executada manualmente, pela linha de comando (CLI) dos equipamentos, seguindo um procedimento documentado.

O processo manual é lento, depende da experiência de quem o executa e está sujeito a erros de digitação e a inconsistências entre equipamentos. As limitações da configuração via CLI para automação são reconhecidas há muito tempo pela comunidade de gerência de redes (Schoenwaelder, 2003), e a automação de redes tornou-se prática corrente na operação de infraestruturas de grande porte (Oswalt et al., 2023).

Este trabalho segue a linha de Projeto: seu foco é a aplicação estruturada de ferramentas existentes a um ambiente complexo, com avaliação quantitativa dos resultados, e não a proposição de uma ferramenta nova. O objetivo é aplicar ferramentas de automação em Python (Netmiko, NAPALM e Jinja2) à ativação de clientes corporativos em roteadores Cisco, em um laboratório que reproduz a arquitetura de um ISP, e medir o ganho em relação ao processo manual quanto a tempo, erros, capacidade de detectar falhas e escalabilidade.

Os resultados mostram redução de 96,5% no tempo médio de ativação, ausência de erros no fluxo automatizado, detecção de 100% dos erros injetados, crescimento linear do tempo com o número de clientes e, na auditoria de conformidade, detecção de todos os desvios em cerca de 4 s, contra 78% em quase seis minutos na auditoria manual. Na falha de um roteador de borda, o fluxo ativou na contingência os clientes com acesso redundante e não aplicou nenhuma configuração ao cliente sem redundância. A Seção 2 apresenta o contexto da organização e o diagnóstico do processo atual; a Seção 3 descreve as tecnologias e os protocolos envolvidos; a Seção 4 detalha a proposta, o ambiente, o protocolo experimental, os resultados e as limitações; a Seção 5 lista as referências, e o Apêndice A reúne os registros dos experimentos.

2. Contexto e Diagnóstico

2.1. A organização

A organização considerada é um provedor de serviços de Internet de grande porte, com atuação nacional, aqui tratado como PROVEDOR. O PROVEDOR oferece acesso em banda larga a clientes residenciais, conectividade IP dedicada a clientes corporativos e troca de tráfego com outras redes (trânsito e *peering*). Sua infraestrutura é um *backbone* IP com roteadores de *core* e de borda, em que os roteadores de borda concentram as conexões dos clientes.

Participam da ativação de um cliente corporativo as áreas comercial e de atendimento corporativo, que registram a contratação; a engenharia de redes, que define os padrões de configuração; a equipe de provisionamento, que executa a ativação; e o centro de operações de rede (NOC), que monitora o serviço após a entrega. Os sistemas de TI envolvidos são, de forma típica, o sistema comercial e de ordens de serviço, o inventário

de rede e de endereçamento, a plataforma de monitoramento, o controle de acesso aos equipamentos (AAA) e a coleta de registros (*syslog*).

2.2. Diagnóstico do processo atual

A ativação é feita manualmente: a partir da ordem de serviço, um técnico acessa o roteador de borda por SSH, digita os comandos de configuração do cliente seguindo o procedimento padrão, verifica o resultado com comandos de consulta e registra a conclusão. Esse processo apresenta cinco problemas principais:

- **Tempo e dependência do operador:** cada ativação consome minutos de trabalho especializado, e o tempo varia conforme a experiência e o estado de quem executa.
- **Erros silenciosos:** um comando sintaticamente válido, mas com o valor errado, é aceito pelo roteador sem nenhum aviso. Durante a montagem do laboratório deste trabalho, a configuração manual do OSPF em apenas quatro roteadores produziu um erro desse tipo: no roteador de borda 3 foi declarada a rede da interface de gerência do roteador de borda 2, e a falha só foi percebida na verificação da tabela de rotas do *core*.
- **Verificação não sistemática:** a conferência depende de o operador lembrar de executá-la, no equipamento certo e com o critério certo.
- **Rastreabilidade limitada:** reconstruir quem alterou o quê, e quando, exige consultar registros dispersos.
- **Escala:** o esforço cresce na proporção direta do número de ativações, sem ganho de produtividade.

2.3. Oportunidade e justificativa

A oportunidade identificada é automatizar a ativação com ferramentas de código aberto que operam sobre a CLI já existente, sem exigir a substituição de equipamentos nem recursos novos nos roteadores. A proposta mantém o procedimento padrão, mas o executa por meio de um fluxo que gera a configuração a partir de dados estruturados, valida os dados antes de tocar na rede, verifica o resultado depois da mudança e desfaz a configuração quando a verificação falha.

O estudo delimita-se à ativação de clientes na camada IP/BGP, a tarefa mais frequente na borda. Serviços como MPLS e QoS operam sobre o mesmo fluxo de automação, variando apenas os modelos de configuração, e ficam indicados como extensões. A implementação foi feita em laboratório, e não na rede de produção do PROVEDOR.

3. Estado da Arte

3.1. Automação e programabilidade de redes

A automação de redes substitui a execução manual de comandos por processos repetíveis, descritos em código e alimentados por dados estruturados, o que reduz o erro humano e padroniza as configurações (Oswalt et al., 2023). Um dos marcos dessa discussão é o relatório do *workshop* de gerência de redes do IAB, que apontou a dificuldade de automatizar equipamentos por meio de interfaces de linha de comando pensadas para uso humano (Schoenwaelder, 2003).

Como resposta, surgiram abordagens orientadas a modelos, como o protocolo NETCONF (Enns et al., 2011) e a linguagem de modelagem YANG (Bjorklund, 2016), que permitem configurar e consultar equipamentos com dados estruturados. Em redes com grande parque instalado, porém, a automação sobre a CLI continua amplamente usada, porque funciona com os equipamentos existentes. Este trabalho adota essa abordagem e mantém a arquitetura independente do meio de transporte, o que permite evoluir para NETCONF/YANG.

3.2. Ferramentas utilizadas

Foram usadas três bibliotecas em Python, cada uma com um papel definido (Tabela 1). O **Netmiko** estabelece sessões SSH com equipamentos de rede e trata os detalhes da CLI, como os diferentes modos de configuração e os tempos de resposta (Byers, 2026). O **NAPALM** oferece uma interface única, independente de fabricante, para ler informações dos equipamentos na forma de dados estruturados (NAPALM, 2026). O **Jinja2** é um mecanismo de modelos que gera texto, neste caso a configuração, a partir de um modelo com campos variáveis (Pallets, 2026). Os dados dos clientes e o inventário ficam em arquivos YAML, que funcionam como fonte da verdade.

Tabela 1. Papel de cada ferramenta no fluxo de automação

Ferramenta	Papel no trabalho
YAML	Fonte da verdade: dados dos clientes e inventário dos roteadores
Jinja2	Geração da configuração padronizada a partir de modelos
Netmiko	Aplicação da configuração e execução de comandos de verificação via SSH
NAPALM	Leitura estruturada: inventário (get_facts) e backup (get_config)

Os arquivos YAML fazem o papel de **fonte da verdade** (*source of truth*): o registro oficial do estado que a rede deve ter, a partir do qual a configuração é gerada e contra o qual ela é conferida. Em operação real, esse papel costuma ser exercido por um sistema de inventário e endereçamento, como o NetBox, ferramenta de código aberto para documentação de equipamentos, circuitos e endereços IP, consultada pelos scripts por

meio de uma API (NetBox Labs, 2026). Neste trabalho, o YAML versionado em Git cumpre a mesma função em escala de laboratório.

Existem alternativas conhecidas para automação de redes, resumidas na Tabela 2. Ansible e Nornir são ferramentas de orquestração: organizam a execução de tarefas sobre um inventário de equipamentos. O Nornir é um *framework* em Python que usa o Netmiko e o NAPALM como *plugins* de conexão e o Jinja2 para modelos (Nornir, 2026); o Ansible descreve as tarefas em arquivos YAML (*playbooks*) e também usa o Jinja2 para gerar configurações, com módulos próprios para equipamentos de rede (Red Hat, 2026). Em ambos os casos, a lógica de uma mudança continua a mesma: gerar a configuração, aplicá-la e verificá-la.

Tabela 2. Alternativas consideradas e motivo da escolha

Alternativa	O que é	Por que não foi a base deste trabalho
Ansible	Orquestrador declarativo, sem agente, com *playbooks* em YAML	Esconde as etapas em módulos e condicionais; a verificação e o *rollback* ficam menos explícitos e mais difíceis de medir separadamente
Nornir	*Framework* Python de orquestração, com execução em paralelo	Usa as mesmas bibliotecas deste trabalho; acrescenta paralelismo e inventário, com ganho pequeno em quatro roteadores. É a evolução natural do fluxo
Cisco NSO	Plataforma comercial de orquestração orientada a modelos, com transações	Licença e porte incompatíveis com o escopo e com a proposta de custo zero
NETCONF/ YANG	Configuração por modelos de dados estruturados	Exige suporte e habilitação nos equipamentos; o cenário estudado é de parque existente operado por CLI

A escolha por aplicar as bibliotecas diretamente seguiu quatro critérios: custo zero e código aberto; compatibilidade com os equipamentos já instalados, via SSH, sem exigir recursos novos; maturidade e comunidade ativa; e transparência de cada etapa no código. Esse último critério é o mais relevante para a avaliação: como o fluxo não fica escondido dentro de uma plataforma, é possível medir separadamente a validação, a aplicação e a verificação, e mostrar exatamente onde o tempo é gasto e onde cada erro é barrado. As três bibliotecas são a camada de base sobre a qual ferramentas como o Nornir são construídas; por isso, o fluxo desenvolvido pode migrar para um orquestrador sem mudar sua lógica.

3.3. Protocolos do ambiente

O ambiente usa o OSPF (Moy, 1998) como protocolo de roteamento interno, responsável pela alcançabilidade da infraestrutura, e o BGP-4 (Rekhter et al., 2006) para anunciar os prefixos dos clientes. Em uma sessão iBGP, um roteador não repassa a outros vizinhos iBGP as rotas aprendidas de um vizinho iBGP; por isso, sem

mecanismos adicionais, todos os roteadores precisariam de sessões entre si, o que exige $n(n-1)/2$ sessões. O Route Reflector (Bates et al., 2006) resolve esse problema: o roteador *core* reflete as rotas entre os roteadores de borda, que mantêm uma única sessão cada. A sincronização de horário entre os equipamentos usa o NTP (Mills et al., 2010), necessário para que os registros de diferentes equipamentos sejam comparáveis.

A separação de funções entre OSPF e BGP é prática comum em provedores. O protocolo interno transporta apenas a infraestrutura (interfaces de gerência e enlaces entre roteadores), o que o mantém pequeno e com convergência rápida. Os prefixos dos clientes são transportados pelo BGP, projetado para tabelas grandes e para a aplicação de políticas de roteamento (Rekhter et al., 2006). O iBGP distribui esses prefixos dentro do sistema autônomo: leva o prefixo do roteador de borda que atende o cliente a todos os demais, inclusive ao roteador de *peering*, que o anunciaria a outras redes por eBGP. As sessões iBGP são estabelecidas entre *loopbacks*, que continuam alcançáveis pelo OSPF mesmo quando um enlace físico falha. No fluxo proposto, a presença do prefixo do cliente no BGP do Route Reflector é a evidência objetiva de que a ativação se propagou pela rede, e por isso a verificação é feita nele.

3.4. Ambiente de simulação

O laboratório foi construído no Cisco Modeling Labs (CML), plataforma que executa imagens virtuais de sistemas operacionais de rede da Cisco (Cisco Systems, 2026). Foi usada a versão gratuita, limitada a cinco nós ativos.

4. Estudo de Caso

4.1. Ambiente de laboratório

A topologia reproduz, em escala reduzida, a arquitetura de *core* e borda de um ISP (Figura 1): um roteador *core*, que atua como Route Reflector; três roteadores de borda, com funções distintas (clientes corporativos, clientes residenciais e *peering*); e um servidor Linux de automação. Os roteadores usam a imagem IOL-XE, com IOS XE 17.18.2. O OSPF área 0 garante a alcançabilidade das interfaces de gerência (*loopbacks*) e o iBGP, estabelecido entre *loopbacks*, transporta apenas os prefixos dos clientes.

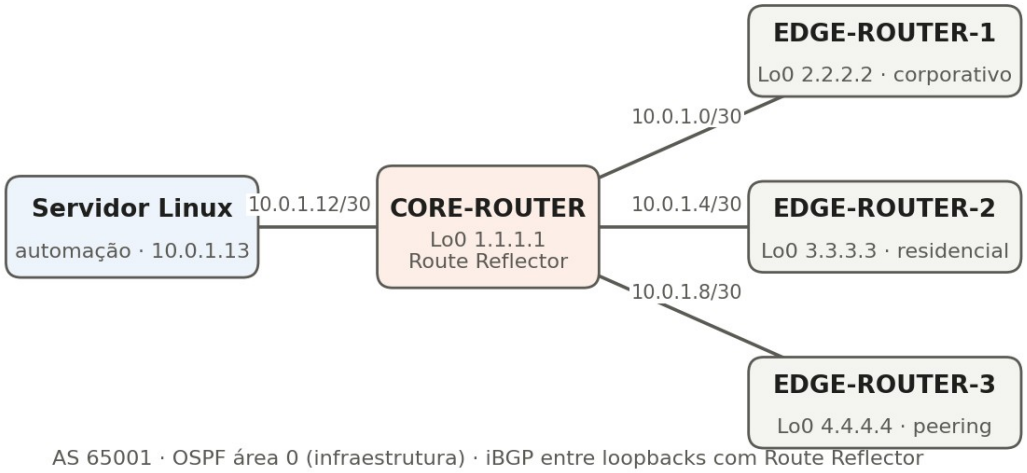


Figura 1. Topologia do laboratório no Cisco Modeling Labs

O servidor Linux (Ubuntu 24.04) executa os scripts em um ambiente virtual Python com Netmiko 4.7.0 e NAPALM 5.2.0. As credenciais são lidas de variáveis de ambiente, e nenhuma senha fica gravada no código. Como o limite de cinco nós impede incluir o roteador do cliente, o ponto de entrega do serviço é representado por uma interface *Loopback* no roteador de borda, e o anúncio do prefixo no BGP é idêntico ao de um cliente real.

Para garantir a rastreabilidade, todos os equipamentos sincronizam o horário com o roteador *core* via NTP, e os roteadores registram cada comando de configuração com horário, usuário e forma de acesso (console ou SSH). O código-fonte, os dados e os registros dos experimentos estão versionados em um repositório Git.

4.2. Proposta: fluxo de ativação

A ativação de um cliente segue um fluxo de cinco etapas (Figura 2). Na **validação**, os dados do cliente são conferidos sem acessar a rede: campos obrigatórios, prefixo IPv4 válido e roteador existente no inventário. No **pré-check**, o script consulta a rede para saber se o prefixo já está em uso ou se o cliente já foi ativado. Na **geração e aplicação**, o Jinja2 monta os comandos e o Netmiko os envia ao roteador de borda. Na **verificação**, o script confere no Route Reflector se o prefixo chegou ao BGP e se o gateway do cliente responde ao ping. Na **decisão**, a configuração é salva se a verificação passar, ou desfeita automaticamente (*rollback*) se falhar.

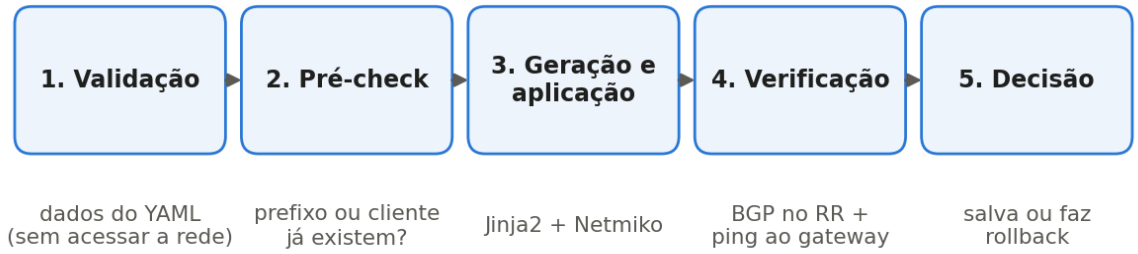


Figura 2. Fluxo de ativação automatizada em cinco etapas

A configuração gerada para um cliente contém a interface de entrega com descrição padronizada (cliente, circuito e velocidade), o endereço do gateway e o anúncio do prefixo no BGP:

```
interface Loopback101
 description CLIENTE:EMPRESA-ALFA | CIRCUITO:CORP-0101 | VEL:100M
 ip address 172.16.101.1 255.255.255.0
router bgp 65001
 network 172.16.101.0 mask 255.255.255.0
```

4.3. Protocolo experimental

Foram definidos cinco experimentos, identificados de D1 a D5, cada um respondendo a uma pergunta sobre a proposta (Tabela 3). Dentro dos experimentos, os identificadores seguem a mesma lógica: R indica a repetição ou rodada, E indica um erro injetado (D2), L indica um lote (D3), C indica uma regra de conformidade e A, um aviso (D4).

Tabela 3. Experimentos e perguntas que respondem

ID	Pergunta	Medição
D1	A automação é mais rápida e erra menos que o processo manual?	Tempo por ativação e erros, em 10 repetições pareadas
D2	O fluxo detecta dados errados e falhas de rede?	Erros barrados ou revertidos, em 7 cenários injetados
D3	O fluxo escala com o número de clientes?	Tempo total em lotes de 1, 5 e 10 clientes, 3 repetições
D4	A automação ajuda a manter a rede padronizada?	Desvios detectados em auditoria, com desvios sorteados às cegas
D5	O que acontece quando um roteador de borda falha?	Decisão por cliente: contingência ou falha segura

No D1, cada repetição ativa o mesmo cliente duas vezes: primeiro de forma manual, digitada a partir de uma ficha e cronometrada do início da configuração até a verificação bem-sucedida no *core*; depois pelo script. O D1 foi executado em duas sessões, com o mesmo procedimento: repetições R01 a R05 no roteador de borda 1 e R06 a R10 no roteador de borda 2.

No D5, cinco clientes (111 a 115) são ativados duas vezes: com a rede normal, como controle, e com o roteador de borda 1 isolado, o que simula sua falha por meio do desligamento da interface do *core* que o conecta. Dois clientes da borda 1 têm acesso redundante, com um segundo circuito na borda 3; um tem circuito único; e dois estão na borda 2. Antes de configurar, o script testa se o roteador principal responde na porta SSH, com limite de 5 s, e decide para cada cliente: ativar no principal; ativar no roteador de contingência, se houver acesso redundante; ou registrar falha segura, sem aplicar nenhuma configuração.

No D4, um script lê a configuração dos quatro roteadores via NAPALM e a confere contra oito regras do padrão (Tabela 4). Em cada rodada, o script gera e aplica de dois a quatro desvios, escolhidos ao acaso entre 20 combinações de roteador e regra e com variações na forma de cada erro, sem revelar quantos são nem onde estão. Em seguida, o operador faz a auditoria manual, cronometrada, registrando cada desvio encontrado; depois, o script faz a auditoria automatizada; por fim, o gabarito é revelado, as duas auditorias são comparadas e os desvios são revertidos. Foram feitas três rodadas. Uma rodada piloto, com um catálogo fixo de desvios, revelou ambiguidade na lista de verificação entregue ao operador e foi descartada; a lista foi corrigida antes das rodadas válidas.

Tabela 4. Regras de conformidade auditadas no D4

ID	Regra	Aplica-se a
C01	Resolução de nomes desativada (`no ip domain lookup`)	Todos
C02	Registro local de eventos (`logging buffered 64000`)	Todos
C03	Registros com data e milissegundos	Todos
C04	Usuário administrativo com privilégio 15	Todos
C05	Acesso remoto com autenticação local	Todos
C06	Acesso remoto somente por SSH	Todos
C07	*Core* como servidor de horário (NTP)	*Core*
C08	Interfaces de cliente com descrição padronizada	Bordas

4.4. Resultados

D1: manual × automatizado. A ativação automatizada levou em média 5,7 s, contra 162,0 s do processo manual, uma redução de 96,5% (Tabela 5 e Figura 3). Pela mediana, menos sensível às repetições com problemas, a redução foi de 94,7%. Houve cinco erros em três das dez repetições manuais e nenhum no fluxo automatizado.

Tabela 5. Resultados do experimento D1 (10 repetições)

Métrica	Manual	Automatizado
Tempo médio (s)	162,0	5,7
Mediana (s)	105,9	5,6
Desvio padrão (s)	124,5	0,4
Tempo das 10 ativações	27,0 min	57,1 s
Repetições com erro	3 de 10	0 de 10
Configuração final correta	10 de 10	10 de 10

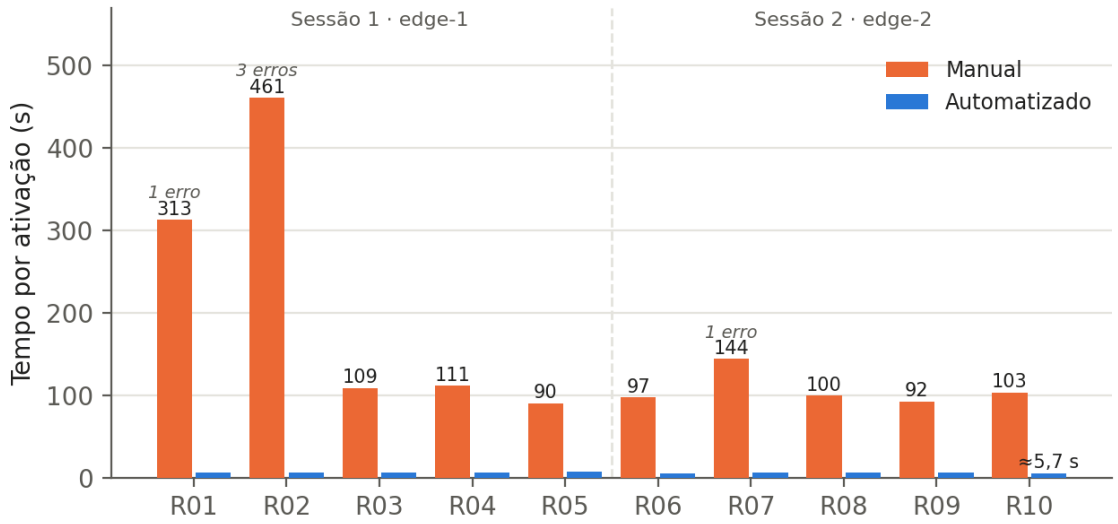


Figura 3. Tempo por ativação em cada repetição do D1

Os erros manuais foram de três tipos: configuração não aplicada, percebida apenas na verificação (R01); configuração aplicada no roteador errado (R02), situação em que a verificação manual apresentou um falso positivo, pois o ping respondeu a partir do próprio *core*; e verificação feita no equipamento errado (R07). Nenhum deles é possível no fluxo automatizado, que sempre configura o roteador indicado no inventário e verifica no Route Reflector.

D2: erros injetados. O fluxo tratou corretamente os sete cenários (Tabela 6). Quatro erros de dados foram barrados na validação, sem acesso à rede; dois conflitos com o estado da rede foram barrados no pré-check; e a falha de rede, simulada com a sessão BGP do roteador de borda derrubada, foi detectada na verificação, com *rollback* automático confirmado nos roteadores.

Tabela 6. Resultados do experimento D2

ID	Erro injetado	Barrado em	Tempo (s)
E1	Prefixo com octeto inválido	Validação	0,00
E2	Prefixo com bits de host	Validação	0,00
E3	Roteador inexistente	Validação	0,00
E4	Prefixo já usado por outro cliente	Pré-check	1,11
E5	Campo obrigatório ausente	Validação	0,00
E6	Sessão BGP da borda derrubada	Verificação + rollback	51,96
E7	Cliente ativado duas vezes	Pré-check	1,11

D3: escala em lotes. As 48 ativações dos nove lotes foram concluídas com sucesso. O tempo por cliente permaneceu estável entre 5,5 s e 6,4 s, e o tempo total cresceu de forma linear, na razão de cerca de 5,7 s por cliente ($R^2 = 0,97$), o que projeta cerca de 9,5 minutos para 100 ativações (Figura 4).

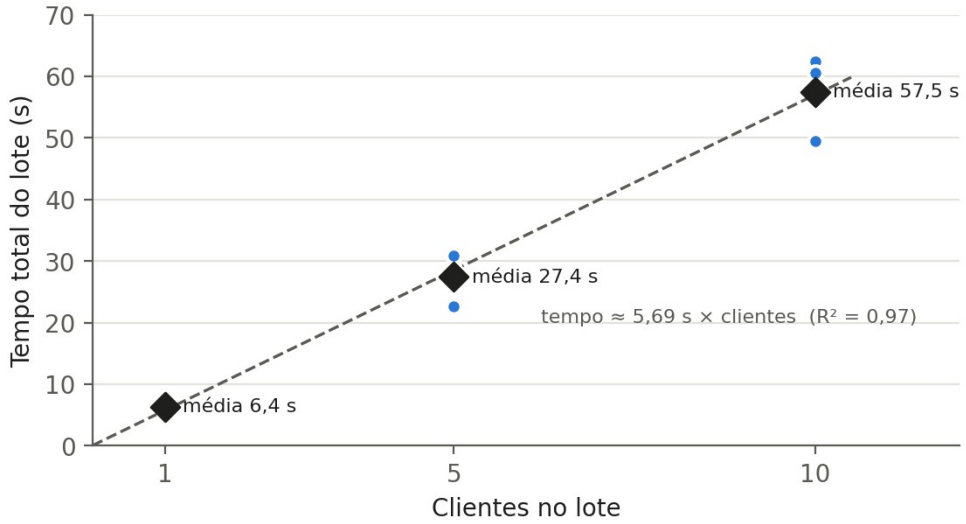


Figura 4. Tempo total por tamanho de lote no experimento D3

D4: auditoria de conformidade. Nas três rodadas foram injetados nove desvios. A auditoria automatizada detectou os nove, sem falsos positivos, em cerca de 4,3 s por rodada. A auditoria manual detectou sete, também sem falsos positivos, com média de 346,1 s e mediana de 371,2 s, ou seja, cerca de 80 vezes mais lenta (Tabela 7). Os dois desvios não encontrados estavam visíveis nas saídas consultadas pelo operador: uma interface de cliente sem descrição e o acesso remoto por Telnet liberado no roteador *core*. A linha de base, antes dos desvios, apontou um ponto de atenção real de segurança: o servidor HTTP habilitado nos quatro roteadores.

Tabela 7. Resultados do experimento D4 (3 rodadas)

Rodada	Desvios	Manual: detectados / tempo	Automatizado: detectados / tempo
1	3	2 de 3 / 371,2 s	3 de 3 / 4,4 s
2	4	3 de 4 / 405,6 s	4 de 4 / 4,2 s
3	2	2 de 2 / 261,5 s	2 de 2 / 4,3 s
Total	**9**	**7 de 9 (78%) / média 346,1 s**	**9 de 9 (100%) / média 4,3 s**

D5: contingência em falha de borda. No controle, os cinco clientes foram ativados no roteador principal, entre 3,3 s e 3,8 s cada. Com a borda 1 isolada, a falha foi detectada de imediato, porque o *core* respondeu com a mensagem de destino inalcançável, sem que fosse preciso esperar o limite de 5 s. Os dois clientes com acesso redundante foram ativados na borda 3 e verificados no Route Reflector, com próximo salto 4.4.4.4; o cliente de circuito único recebeu falha segura, sem nenhuma configuração aplicada, o que foi confirmado no *core* (`Network not in table`) e na própria borda 1 após o restabelecimento; e os clientes da borda 2 foram ativados normalmente (Tabela 8). Nenhum cliente ficou com configuração parcial.

Tabela 8. Resultados do experimento D5

Cliente	Acesso	Controle (rede normal)	Falha da borda 1
111	Redundante (bordas 1 e 3)	Principal (borda 1), OK, 3,75 s	Contingência (borda 3), OK, 3,72 s
112	Redundante (bordas 1 e 3)	Principal (borda 1), OK, 3,40 s	Contingência (borda 3), OK, 3,66 s
113	Circuito único (borda 1)	Principal (borda 1), OK, 3,38 s	Falha segura: nada aplicado
114	Circuito único (borda 2)	Principal (borda 2), OK, 3,34 s	Principal (borda 2), OK, 6,87 s
115	Circuito único (borda 2)	Principal (borda 2), OK, 3,76 s	Principal (borda 2), OK, 6,71 s

4.5. Discussão e limitações

A configuração em si leva cerca de três segundos; o restante do tempo automatizado é a espera pela convergência da rede e pela verificação, ou seja, a automação não dá o serviço como ativo antes de confirmá-lo. Nas repetições manuais houve curva de aprendizado: as duas primeiras tiveram média de 386,9 s, e as oito seguintes, 105,8 s. Ainda assim, mesmo sem erros, o processo manual foi cerca de 18 vezes mais lento. A variação também difere: 0,4 s de desvio padrão no automatizado, contra 124,5 s no manual.

No D4, a diferença não foi só de tempo. A auditoria manual depende de atenção a dezenas de linhas por roteador, e os dois desvios perdidos estavam na tela. A rodada

piloto mostrou ainda que o resultado manual depende de como o operador interpreta o padrão escrito, enquanto o script aplica a regra sempre da mesma forma.

No D5, a decisão de onde ativar cada cliente veio de um dado cadastrado, o roteador de contingência no YAML, e não de uma escolha do operador sob pressão durante a falha. O tempo maior dos clientes da borda 2 no cenário de falha (6,7 s a 6,9 s) é compatível com o grupo mais lento observado no D3, em que o prefixo ainda não havia chegado ao *core* na primeira consulta e o script aguardou 3 s para consultar de novo.

O trabalho tem limitações que devem ser consideradas. O limite de cinco nós do CML Free impede incluir o roteador do cliente e um sistema autônomo externo. Os roteadores são imagens virtuais, e não os equipamentos de alta capacidade de um ISP. As medições manuais foram realizadas por um único operador, responsável também pelo desenvolvimento da automação; por conhecer bem o procedimento, seus tempos representam o melhor caso do processo manual, e a vantagem medida da automação tende a ser conservadora; no D4, o operador já conhecia o conjunto de regras. As regras C04 e C05 são auditadas, mas não foram usadas como desvios, porque alterá-las cortaria o acesso do próprio script aos roteadores. O NTP do laboratório opera sem autenticação, o que seria necessário em produção. No D5, a falha foi simulada em um único cenário, pelo desligamento de uma interface, e o acesso redundante do cliente foi representado pelo cadastro do roteador de contingência, já que o laboratório não inclui o roteador do cliente com dois circuitos.

4.6. Considerações finais

O objetivo deste trabalho foi aplicar Netmiko, NAPALM e Jinja2 à ativação de clientes corporativos em um ambiente que reproduz a arquitetura de um ISP e medir o ganho em relação ao processo manual. O objetivo foi atingido, e as perguntas do protocolo experimental tiveram resposta objetiva. A automação foi mais rápida e não errou: o tempo médio de ativação caiu de 162,0 s para 5,7 s, uma redução de 96,5%, e os cinco erros do processo manual não se repetiram no fluxo automatizado (D1). O fluxo detectou falhas: barrou ou reverteu os sete erros injetados, sem deixar configuração incorreta nos roteadores (D2). O fluxo escalou: o tempo cresceu de forma linear, cerca de 5,7 s por cliente (D3). E a automação ajudou a manter a rede padronizada: a auditoria automatizada encontrou os nove desvios injetados em cerca de 4 s, enquanto a manual encontrou sete em quase seis minutos (D4). Por fim, o fluxo se comportou de forma segura diante da falha de uma borda: ativou na contingência os clientes com acesso redundante, não aplicou nada ao cliente sem redundância e manteve normais os clientes das demais bordas (D5).

Mais do que a velocidade, o ganho vem de três características do fluxo. A primeira é a padronização: a configuração é gerada a partir de dados estruturados, e erros de digitação ou de equipamento deixam de ser possíveis. A segunda é a verificação sistemática: nenhuma ativação é dada como concluída sem confirmação no Route

Reflector, o que elimina o falso positivo observado no processo manual. A terceira é o *rollback*: quando a verificação falha, a mudança é desfeita sem depender da atenção do operador. A auditoria mostrou o mesmo padrão em outra tarefa: o operador deixou passar desvios que estavam na tela, e o script aplicou as regras sempre da mesma forma.

Para o PROVEDOR, a proposta tem aplicação direta, porque funciona sobre o parque existente, via SSH, sem troca de equipamentos e sem custo de licença. Uma adoção gradual pode começar pela auditoria de conformidade, que apenas lê as configurações e tem baixo risco, passar à ativação com verificação e *rollback* e, depois, integrar o fluxo ao processo de gestão de mudanças, em que os dados versionados em Git registram o que foi solicitado e os registros de execução comprovam o que foi feito.

Os resultados devem ser lidos à luz das limitações descritas na Seção 4.5: ambiente simulado com cinco nós, um único operador no processo manual e um conjunto restrito de regras de auditoria. Essas limitações indicam os caminhos de continuidade. A orquestração com Nornir permitiria executar as tarefas em paralelo sobre centenas de roteadores, e o Ansible facilitaria o uso compartilhado por equipes. A adoção do NetBox como fonte da verdade substituiria os arquivos YAML por um inventário central consultado via API. O uso de NETCONF/YANG trocaria a leitura de texto da CLI por dados estruturados nos equipamentos que o suportam. A validação prévia das mudanças em um *pipeline* de integração contínua permitiria testar cada alteração antes de aplicá-la. Por fim, o mesmo fluxo pode ser estendido a serviços MPLS e a sessões de *peering* eBGP, com novos modelos de configuração e a mesma lógica de validação, verificação e *rollback*.

5. Referências Bibliográficas

Bates, T., Chen, E. and Chandra, R. (2006) “BGP Route Reflection: An Alternative to Full Mesh Internal BGP (IBGP)”, RFC 4456, IETF.

Bjorklund, M. (2016) “The YANG 1.1 Data Modeling Language”, RFC 7950, IETF.

Byers, K. (2026) “Netmiko: multi-vendor library to simplify CLI connections to network devices”, https://github.com/ktbyers/netmiko, acesso em setembro de 2026.

Cisco Systems (2026) “Cisco Modeling Labs v2.10 Documentation”, https://developer.cisco.com/docs/modeling-labs/, acesso em setembro de 2026.

Enns, R., Bjorklund, M., Schoenwaelder, J. and Bierman, A. (2011) “Network Configuration Protocol (NETCONF)”, RFC 6241, IETF.

Mills, D., Martin, J., Burbank, J. and Kasch, W. (2010) “Network Time Protocol Version 4: Protocol and Algorithms Specification”, RFC 5905, IETF.

Moy, J. (1998) “OSPF Version 2”, RFC 2328, IETF.

NAPALM (2026) “NAPALM Documentation”, https://napalm.readthedocs.io/, acesso em setembro de 2026.

NetBox Labs (2026) “NetBox Documentation”, https://netboxlabs.com/docs/netbox/, acesso em setembro de 2026.

Nornir (2026) “Nornir Documentation”, https://nornir.readthedocs.io/, acesso em setembro de 2026.

Oswalt, M., Adell, C., Lowe, S. S. and Edelman, J. (2023) Network Programmability and Automation, O’Reilly Media, 2nd edition.

Pallets (2026) “Jinja Documentation”, https://jinja.palletsprojects.com/, acesso em setembro de 2026.

Red Hat (2026) “Ansible Documentation”, https://docs.ansible.com/, acesso em setembro de 2026.

Rekhter, Y., Li, T. and Hares, S. (2006) “A Border Gateway Protocol 4 (BGP-4)”, RFC 4271, IETF.

Schoenwaelder, J. (2003) “Overview of the 2002 IAB Network Management Workshop”, RFC 3535, IETF.

Apêndice A – Registros dos experimentos

Este apêndice reúne os registros de execução dos experimentos D1 a D5. As saídas foram copiadas dos terminais do servidor de automação e dos consoles dos roteadores. Para preservar a segurança, o *hash* de senha exibido pelo comando de usuário foi substituído por [HASH REMOVIDO]. Os horários do servidor Linux estão em BRT (UTC−3) e os dos roteadores em UTC, com os relógios sincronizados por NTP. Os arquivos CSV completos e as saídas integrais estão no repositório do projeto.

A.1. D1 – Ativação manual × automatizada

Tabela A.1. Tempo por repetição no D1 (manual cronometrado; automatizado registrado pelo script)

Repetição	Cliente	Roteador	Manual (s)	Erros	Automatizado (s)
R01	101	borda 1	312,85	1	5,57
R02	102	borda 1	460,97	3	5,66
R03	103	borda 1	108,57	0	5,55
R04	104	borda 1	111,29	0	5,54
R05	105	borda 1	90,19	0	6,88
R06	106	borda 2	97,16	0	5,42
R07	107	borda 2	143,76	1	5,69
R08	108	borda 2	99,70	0	5,77
R09	109	borda 2	92,22	0	5,55
R10	110	borda 2	103,17	0	5,47
Média / total			**162,0**	**5**	**5,7**

Falhas do processo manual: na R01, a configuração não foi aplicada e a falha só apareceu na verificação do *core* (`Network not in table`, ping 0%); na R02, o cliente foi configurado primeiro no *core* e depois na borda 2 (roteador errado duas vezes), com falso positivo na verificação (o *core* respondia ao ping da própria interface) e um erro de digitação; na R07, a verificação foi feita na borda 1 em vez do *core*. Em todas as repetições, a trilha de auditoria dos roteadores (`show archive log config all`) registrou os comandos manuais (console) separados das sessões do script (SSH).

Exemplo de execução do fluxo automatizado (cliente 101) e verificação independente nos roteadores:

```
$ python 03_ativar_cliente.py 101
=== cliente-101 EMPRESA-ALFA ===
  Configuracao gerada:
    interface Loopback101
     description CLIENTE:EMPRESA-ALFA | CIRCUITO:CORP-0101 | VEL:100M
     ip address 172.16.101.1 255.255.255.0
     no shutdown
    router bgp 65001
     network 172.16.101.0 mask 255.255.255.0
  [OK] BGP ok; ping 100% (3.7 s)
```

```
$ python 03_ativar_cliente.py 101          (segunda execucao: duplicidade)
=== cliente-101 EMPRESA-ALFA ===
  [REJEITADO] prefixo 172.16.101.0/24 ja esta em uso na rede

EDGE-ROUTER-1#show running-config interface Loopback101
interface Loopback101
 description CLIENTE:EMPRESA-ALFA | CIRCUITO:CORP-0101 | VEL:100M
 ip address 172.16.101.1 255.255.255.0

CORE-ROUTER#show ip bgp 172.16.101.0
BGP routing table entry for 172.16.101.0/24, version 23
  Local, (Received from a RR-client)
    2.2.2.2 (metric 11) from 2.2.2.2 (2.2.2.2)
      Origin IGP, metric 0, localpref 100, valid, internal, best
      Updated on Sep 26 2026 18:41:16 UTC

EDGE-ROUTER-2#show ip bgp 172.16.101.0
BGP routing table entry for 172.16.101.0/24, version 16
  Local
    2.2.2.2 (metric 21) from 1.1.1.1 (1.1.1.1)
      Origin IGP, metric 0, localpref 100, valid, internal, best
      Originator: 2.2.2.2, Cluster list: 1.1.1.1
      Updated on Sep 26 2026 18:41:16 UTC

$ python 04_remover_cliente.py 101
cliente-101: removido de edge-router-1 (1.6 s)
```

A.2. D2 – Erros injetados

Registro do script (`logs/resultados_D2_erros.csv`) em 26/09/2026. A primeira linha é a limpeza da sessão anterior.

```
data_hora,operacao,alvo,duracao_s,resultado,detalhe
2026-09-26T15:48:15,remocao,cliente-101,1.60,OK,
2026-09-26T15:53:38,ativacao,cliente-101,6.59,OK,BGP ok; ping 100%
2026-09-26T15:53:52,ativacao,cliente-101,1.11,REJEITADO_PRECHECK,prefixo 172.16.101.0/24
    ja esta em uso na rede
2026-09-26T15:54:01,ativacao,cliente-201,0.00,REJEITADO_VALIDACAO,prefixo invalido:
'172.16.301.0/24'
    does not appear to be an IPv4 or IPv6 network
2026-09-26T15:54:01,ativacao,cliente-202,0.00,REJEITADO_VALIDACAO,prefixo invalido:
172.16.202.5/24
    has host bits set
2026-09-26T15:54:01,ativacao,cliente-203,0.00,REJEITADO_VALIDACAO,roteador inexistente no
inventario:
    edge-router-9
2026-09-26T15:54:02,ativacao,cliente-204,1.11,REJEITADO_PRECHECK,prefixo 172.16.101.0/24
    ja esta em uso na rede
2026-09-26T15:54:02,ativacao,cliente-205,0.00,REJEITADO_VALIDACAO,campo obrigatorio ausente:
velocidade
2026-09-26T15:54:27,remocao,cliente-101,1.60,OK,
2026-09-26T15:55:46,ativacao,cliente-103,51.96,FALHA_ROLLBACK,prefixo nao apareceu no BGP do
core
    ou gateway sem resposta

(linhas longas quebradas para caber na pagina)
```

Confirmação do *rollback* do cenário E6: `show running-config | include Loopback103` vazio na borda 1 e `Network not in table` no *core* para 172.16.103.0; sessão BGP da borda 1 restaurada em seguida.

A.3. D3 – Escala em lotes

Registro por lote (`logs/resultados_D3_lotes.csv`), em 27/09/2026. O detalhe das 48 ativações está em `logs/resultados_D3_detalhe.csv`, no repositório.

```
repeticao,lote,clientes_ok,duracao_total_s,media_por_cliente_s
1,1,1,6.74,6.74
1,5,5,22.66,4.53
1,10,10,49.44,4.94
2,1,1,7.00,7.00
2,5,5,28.76,5.75
2,10,10,62.40,6.24
3,1,1,5.37,5.37
3,5,5,30.89,6.18
3,10,10,60.64,6.06
```

Regressão linear sobre as nove medições: inclinação de 5,69 s por cliente, intercepto de 0,07 s, $R^2 = 0{,}97$. As 48 ativações individuais tiveram média de 5,68 s (mínimo 3,3 s; máximo 7,1 s).

A.4. D4 – Auditoria de conformidade

Rodada piloto (29/09/2026, descartada): a lista de verificação citava `logging buffered 64000 debugging`, mas o IOS omite `debugging` por ser o nível padrão, e não deixava claro que a regra C08 só se aplica quando existe interface de cliente (`Loopback1xx`). O operador encontrou os três desvios reais em 461,8 s, mas registrou sete falsos positivos causados pela lista. A lista foi corrigida e o gerador de desvios passou a ser aleatório antes das rodadas válidas. Nas saídas abaixo, as linhas [OK] e [AVISO] da auditoria automatizada foram omitidas, bem como a listagem completa da pós-reversão; a íntegra está no repositório.

Linha de base (antes da rodada 1)

```
$ mkdir -p logs/d4_piloto && mv -n logs/resultados_D4_*.csv logs/d4_piloto/
$ md5sum 09_auditoria.py
8b817b80e9529349b28c36c4b5d6bb31  09_auditoria.py
$ python 09_auditoria.py baseline
===== [D4-BASELINE] INICIO - AUDITORIA =====
core-router
  [OK   ] C01 no ip domain lookup
  [OK   ] C02 logging buffered 64000
  [OK   ] C03 timestamps de log com data e ms
  [OK   ] C04 usuario admin com privilege 15
  [OK   ] C05 vty com login local
  [OK   ] C06 vty somente SSH
  [OK   ] C07 core como servidor NTP
  [AVISO] A01 servidor HTTP desabilitado (recomendacao) -> ip http server habilitado
edge-router-1
  [OK   ] C01 no ip domain lookup
  [OK   ] C02 logging buffered 64000
  [OK   ] C03 timestamps de log com data e ms
  [OK   ] C04 usuario admin com privilege 15
  [OK   ] C05 vty com login local
  [OK   ] C06 vty somente SSH
  [OK   ] C08 loopbacks de cliente com descricao padrao
  [AVISO] A01 servidor HTTP desabilitado (recomendacao) -> ip http server habilitado
edge-router-2
  [OK   ] C01 no ip domain lookup
  [OK   ] C02 logging buffered 64000
  [OK   ] C03 timestamps de log com data e ms
  [OK   ] C04 usuario admin com privilege 15
  [OK   ] C05 vty com login local
  [OK   ] C06 vty somente SSH
  [OK   ] C08 loopbacks de cliente com descricao padrao
  [AVISO] A01 servidor HTTP desabilitado (recomendacao) -> ip http server habilitado
edge-router-3
  [OK   ] C01 no ip domain lookup
  [OK   ] C02 logging buffered 64000
  [OK   ] C03 timestamps de log com data e ms
  [OK   ] C04 usuario admin com privilege 15
  [OK   ] C05 vty com login local
  [OK   ] C06 vty somente SSH
  [OK   ] C08 loopbacks de cliente com descricao padrao
  [AVISO] A01 servidor HTTP desabilitado (recomendacao) -> ip http server habilitado
Resumo: 32 verificacoes em 4 roteadores | 0 FALHA(s) | 4 AVISO(s) | 4.1 s
===== [D4-BASELINE] FIM - AUDITORIA =====
```

Rodada 1 – 29/09/2026, início às 23:11 (BRT)

```
$ date
Tue Sep 29 23:11:58 -03 2026
$ python 09_auditoria.py --injetar
===== [D4-DESVIOS-R1] DESVIOS APLICADOS (quantos, quais e onde ficam em segredo ate o --
gabarito) =====

--- Linux: auditoria manual (as cegas) ---
$ python 09_auditoria.py --manual desvios
===== [D4-DESVIOS-R1] AUDITORIA MANUAL =====
Durante a auditoria, registre aqui cada desvio encontrado:
  formato : <roteador> <regra>      ex.: e1 C01
  roteador: core | e1 | e2 | e3     regra: C01 a C08
  apagar  : remove o ultimo registro
  fim     : para o cronometro e grava

Pressione ENTER para INICIAR o cronometro...
[00:00] desvio> core c01
  registrado: core-router C01  (total: 1)
[00:30] desvio> core
  formato invalido (ex.: e1 C01)
[01:44] desvio> core c07
  registrado: core-router C07  (total: 2)
[01:49] desvio> fim

  Tempo total: 371.2 s | desvios apontados: 2
    - core-router C01
    - core-router C07
===== [D4-DESVIOS-R1] MANUAL - registrado em logs/resultados_D4_manual.csv =====

--- Consoles dos roteadores (auditoria manual) ---
CORE-ROUTER#sh run | inc domain
ip domain name isp-lab.local
CORE-ROUTER#sh run | inc buffered
logging buffered 64000
CORE-ROUTER#sh run | inc timestamps
service timestamps debug datetime msec
service timestamps log datetime msec
CORE-ROUTER#sh run | inc username
username admin privilege 15 secret 9 [HASH REMOVIDO]
CORE-ROUTER#sh run | sec vty
line vty 0 4
 exec-timeout 0 0
 activation-character 13
 login local
 transport input ssh
CORE-ROUTER#sh run | inc ntp master
CORE-ROUTER#

EDGE-ROUTER-1#sh clock
02:14:30.328 UTC Wed Sep 30 2026
EDGE-ROUTER-1#sh run | inc domain
no ip domain lookup
ip domain name isp-lab.local
EDGE-ROUTER-1#sh run | inc buffered
logging buffered 64000
EDGE-ROUTER-1#sh run | inc timestamps
service timestamps debug datetime msec
service timestamps log datetime msec
EDGE-ROUTER-1#sh run | inc username
username admin privilege 15 secret 9 [HASH REMOVIDO]
EDGE-ROUTER-1#sh run | sec vty
line vty 0 4
 exec-timeout 0 0
 activation-character 13
 login local
 transport input ssh
EDGE-ROUTER-1#sh tun | sec interface
% Incomplete command before pipe.
```

```
EDGE-ROUTER-1#sh run | sec interface
interface Loopback0
 ip address 2.2.2.2 255.255.255.255
interface Ethernet0/0
 description LINK-TO-CORE
 ip address 10.0.1.2 255.255.255.252
interface Ethernet0/1
 no ip address
 shutdown
interface Ethernet0/2
 no ip address
 shutdown
interface Ethernet0/3
 no ip address
 shutdown

EDGE-ROUTER-2#sh clock
02:16:19.198 UTC Wed Sep 30 2026
EDGE-ROUTER-2#sh run | inc domain
no ip domain lookup
ip domain name isp-lab.local
EDGE-ROUTER-2#sh run | inc buffered
logging buffered 64000
EDGE-ROUTER-2#sh run | inc times
service timestamps debug datetime msec
service timestamps log datetime msec
EDGE-ROUTER-2#sh run | sec vty
line vty 0 4
 exec-timeout 0 0
 activation-character 13
 login local
 transport input ssh
EDGE-ROUTER-2#sh run | sec interface
interface Loopback0
 ip address 3.3.3.3 255.255.255.255
interface Ethernet0/0
 description LINK-TO-CORE
 ip address 10.0.1.6 255.255.255.252
interface Ethernet0/1
 no ip address
 shutdown
interface Ethernet0/2
 no ip address
 shutdown
interface Ethernet0/3
 no ip address
 shutdown

EDGE-ROUTER-3#sh clock
02:17:31.299 UTC Wed Sep 30 2026
EDGE-ROUTER-3#sh run | inc domain
no ip domain lookup
ip domain name isp-lab.local
EDGE-ROUTER-3#sh run | inc buffered
logging buffered 64000
EDGE-ROUTER-3#sh run | inc times
service timestamps debug datetime msec
service timestamps log datetime msec
EDGE-ROUTER-3#sh run | inc username
username admin privilege 15 secret 9 [HASH REMOVIDO]
EDGE-ROUTER-3#sh run | sec vty
line vty 0 4
 exec-timeout 0 0
 activation-character 13
 login local
 transport input ssh
EDGE-ROUTER-3#sh run | sec interface
interface Loopback0
 ip address 4.4.4.4 255.255.255.255
interface Loopback177
```

```
 no ip address
interface Ethernet0/0
 description LINK-TO-CORE
 ip address 10.0.1.10 255.255.255.252
interface Ethernet0/1
 no ip address
 shutdown
interface Ethernet0/2
 no ip address
 shutdown
interface Ethernet0/3
 no ip address
 shutdown

--- Linux: auditoria automatizada ---
$ python 09_auditoria.py desvios
===== [D4-DESVIOS-R1] INICIO - AUDITORIA =====

core-router
  [FALHA] C01 no ip domain lookup
  [FALHA] C07 core como servidor NTP

edge-router-1

edge-router-2

edge-router-3
  [FALHA] C08 loopbacks de cliente com descricao padrao -> sem descricao padrao: Loopback177

Resumo: 32 verificacoes em 4 roteadores | 3 FALHA(s) | 4 AVISO(s) | 4.4 s
===== [D4-DESVIOS-R1] FIM - AUDITORIA =====

--- Linux: gabarito e comparacao ---
$ python 09_auditoria.py --gabarito
===== [D4] GABARITO =====
  core-router    regra C07  ->  no ntp master
  edge-router-3  regra C08  ->  interface Loopback177
  core-router    regra C01  ->  ip domain lookup

===== [D4] RODADA 1 - COMPARACAO COM O GABARITO (3 desvios injetados) =====
  Metodo            Tempo  Acertos  Falsos+  Nao encontrados
  manual           371.2s  2/3            0  edge-router-3 C08
  automatizado       4.4s  3/3            0  -
===== [D4] COMPARACAO - registrada em logs/resultados_D4_comparacao.csv =====

--- Linux: reversao ---
$ python 09_auditoria.py --reverter
===== [D4-DESVIOS-R1] DESVIOS REVERTIDOS =====
$ python 09_auditoria.py pos-reversao
Resumo: 32 verificacoes em 4 roteadores | 0 FALHA(s) | 4 AVISO(s) | 4.4 s
===== [D4-POS-REVERSAO] FIM - AUDITORIA =====
```

Rodada 2 – 30/09/2026, início às 10:41 (BRT)

```
$ date
Wed Sep 30 10:41:01 -03 2026
$ python 02_backup.py
[OK]   core-router: backups/core-router_20260930-104113.cfg (146 linhas)
[OK]   edge-router-1: backups/edge-router-1_20260930-104113.cfg (139 linhas)
[OK]   edge-router-2: backups/edge-router-2_20260930-104113.cfg (140 linhas)
[OK]   edge-router-3: backups/edge-router-3_20260930-104113.cfg (139 linhas)
$ python 09_auditoria.py --injetar
===== [D4-DESVIOS-R2] DESVIOS APLICADOS (quantos, quais e onde ficam em segredo ate o --
gabarito) =====

--- Linux: auditoria manual (as cegas) ---
$ python 09_auditoria.py --manual desvios
===== [D4-DESVIOS-R2] AUDITORIA MANUAL =====
Durante a auditoria, registre aqui cada desvio encontrado:
  formato : <roteador> <regra>      ex.: e1 C01
  roteador: core | e1 | e2 | e3     regra: C01 a C08
  apagar  : remove o ultimo registro
  fim     : para o cronometro e grava

Pressione ENTER para INICIAR o cronometro...
[00:00] desvio> e1 c01
  registrado: edge-router-1 C01  (total: 1)
[01:12] desvio> e3 c02
  registrado: edge-router-3 C02  (total: 2)
[05:29] desvio> e3 c03
  registrado: edge-router-3 C03  (total: 3)
[06:03] desvio> fim

  Tempo total: 405.6 s | desvios apontados: 3
    - edge-router-1 C01
    - edge-router-3 C02
    - edge-router-3 C03
===== [D4-DESVIOS-R2] MANUAL - registrado em logs/resultados_D4_manual.csv =====

--- Consoles dos roteadores (auditoria manual) ---
CORE-ROUTER#sh clock
13:40:29.856 UTC Wed Sep 30 2026
CORE-ROUTER#sh run | inc domain
no ip domain lookup
ip domain name isp-lab.local
CORE-ROUTER#sh run | inc buffered
logging buffered 64000
CORE-ROUTER#sh run | inc timestamps
service timestamps debug datetime msec
service timestamps log datetime msec
CORE-ROUTER#sh run | inc username
username admin privilege 15 secret 9 [HASH REMOVIDO]
CORE-ROUTER#sh run | sec vty
line vty 0 4
 exec-timeout 0 0
 activation-character 13
 login local
 transport input telnet ssh
CORE-ROUTER#sh run | inc ntp
ntp master 3

EDGE-ROUTER-1#sh clock
13:42:53.612 UTC Wed Sep 30 2026
EDGE-ROUTER-1#sh run | inc domain
ip domain name isp-lab.local
EDGE-ROUTER-1#sh run | inc buffer
logging buffered 64000
EDGE-ROUTER-1#sh run | inc times
service timestamps debug datetime msec
service timestamps log datetime msec
EDGE-ROUTER-1#sh run | inc username
username admin privilege 15 secret 9 [HASH REMOVIDO]
```

```
EDGE-ROUTER-1#sh run | inc vty
line vty 0 4
EDGE-ROUTER-1#sh run | sec vty
line vty 0 4
 exec-timeout 0 0
 activation-character 13
 login local
 transport input ssh
EDGE-ROUTER-1#sh run | sec interface
interface Loopback0
 ip address 2.2.2.2 255.255.255.255
interface Ethernet0/0
 description LINK-TO-CORE
 ip address 10.0.1.2 255.255.255.252
interface Ethernet0/1
 no ip address
 shutdown
interface Ethernet0/2
 no ip address
 shutdown
interface Ethernet0/3
 no ip address
 shutdown

EDGE-ROUTER-2#sh clock
13:46:00.599 UTC Wed Sep 30 2026
EDGE-ROUTER-2#sh run | inc domain
no ip domain lookup
ip domain name isp-lab.local
EDGE-ROUTER-2#sh run | inc buffere
logging buffered 64000
EDGE-ROUTER-2#sh run | inc times
service timestamps debug datetime msec
service timestamps log datetime msec
EDGE-ROUTER-2#sh run | inc username
username admin privilege 15 secret 9 [HASH REMOVIDO]
EDGE-ROUTER-2#sh run | sec vty
line vty 0 4
 exec-timeout 0 0
 activation-character 13
 login local
 transport input ssh
EDGE-ROUTER-2#sh run | sec interface
interface Loopback0
 ip address 3.3.3.3 255.255.255.255
interface Ethernet0/0
 description LINK-TO-CORE
 ip address 10.0.1.6 255.255.255.252
interface Ethernet0/1
 no ip address
 shutdown
interface Ethernet0/2
 no ip address
 shutdown
interface Ethernet0/3
 no ip address
 shutdown

EDGE-ROUTER-3#sh clock
13:46:48.274 UTC Wed Sep 30 2026
EDGE-ROUTER-3#sh run | inc domain
no ip domain lookup
ip domain name isp-lab.local
EDGE-ROUTER-3#sh run | inc buffered
EDGE-ROUTER-3#sh run | inc buffer
EDGE-ROUTER-3#sh run | inc timest
service timestamps debug datetime msec
service timestamps log uptime
EDGE-ROUTER-3#sh run | inc timestamp
service timestamps debug datetime msec
```

```
service timestamps log uptime
EDGE-ROUTER-3#sh run | inc username
username admin privilege 15 secret 9 [HASH REMOVIDO]
EDGE-ROUTER-3#sh run | sec vty
line vty 0 4
 exec-timeout 0 0
 activation-character 13
 login local
 transport input ssh
EDGE-ROUTER-3#sh run | sec interface
interface Loopback0
 ip address 4.4.4.4 255.255.255.255
interface Ethernet0/0
 description LINK-TO-CORE
 ip address 10.0.1.10 255.255.255.252
interface Ethernet0/1
 no ip address
 shutdown
interface Ethernet0/2
 no ip address
 shutdown
interface Ethernet0/3
 no ip address
 shutdown

--- Linux: auditoria automatizada ---
$ python 09_auditoria.py desvios
===== [D4-DESVIOS-R2] INICIO - AUDITORIA =====

core-router
  [FALHA] C06 vty somente SSH -> transport input telnet ssh

edge-router-1
  [FALHA] C01 no ip domain lookup

edge-router-2

edge-router-3
  [FALHA] C02 logging buffered 64000
  [FALHA] C03 timestamps de log com data e ms

Resumo: 32 verificacoes em 4 roteadores | 4 FALHA(s) | 4 AVISO(s) | 4.2 s
===== [D4-DESVIOS-R2] FIM - AUDITORIA =====

--- Linux: gabarito e comparacao ---
$ python 09_auditoria.py --gabarito
===== [D4] GABARITO =====
  edge-router-1  regra C01  ->  ip domain lookup
  edge-router-3  regra C02  ->  logging buffered 4096
  edge-router-3  regra C03  ->  service timestamps log uptime
  core-router    regra C06  ->  line vty 0 4 / transport input ssh telnet

===== [D4] RODADA 2 - COMPARACAO COM O GABARITO (4 desvios injetados) =====
  Metodo            Tempo  Acertos  Falsos+  Nao encontrados
  manual           405.6s  3/4            0  core-router C06
  automatizado       4.2s  4/4            0  -
===== [D4] COMPARACAO - registrada em logs/resultados_D4_comparacao.csv =====

--- Linux: reversao ---
$ python 09_auditoria.py --reverter
===== [D4-DESVIOS-R2] DESVIOS REVERTIDOS =====
$ python 09_auditoria.py pos-reversao
Resumo: 32 verificacoes em 4 roteadores | 0 FALHA(s) | 4 AVISO(s) | 4.2 s
===== [D4-POS-REVERSAO] FIM - AUDITORIA =====
```

Rodada 3 – 30/09/2026, início às 10:56 (BRT)

```
$ date
Wed Sep 30 10:56:38 -03 2026
$ python 09_auditoria.py --injetar
===== [D4-DESVIOS-R3] DESVIOS APLICADOS (quantos, quais e onde ficam em segredo ate o --
gabarito) =====

--- Linux: auditoria manual (as cegas) ---
$ python 09_auditoria.py --manual desvios
===== [D4-DESVIOS-R3] AUDITORIA MANUAL =====
Durante a auditoria, registre aqui cada desvio encontrado:
  formato : <roteador> <regra>      ex.: e1 C01
  roteador: core | e1 | e2 | e3     regra: C01 a C08
  apagar  : remove o ultimo registro
  fim     : para o cronometro e grava

Pressione ENTER para INICIAR o cronometro...
[00:00] desvio> core c07
  registrado: core-router C07  (total: 1)
[01:35] desvio> e1 c01
  registrado: edge-router-1 C01  (total: 2)
[01:53] desvio> fim

  Tempo total: 261.5 s | desvios apontados: 2
    - core-router C07
    - edge-router-1 C01
===== [D4-DESVIOS-R3] MANUAL - registrado em logs/resultados_D4_manual.csv =====

--- Consoles dos roteadores (auditoria manual) ---
CORE-ROUTER#sh clock
13:57:04.870 UTC Wed Sep 30 2026
CORE-ROUTER#sh run | inc domain
no ip domain lookup
ip domain name isp-lab.local
CORE-ROUTER#sh run | inc buffer
logging buffered 64000
CORE-ROUTER#sh run | inc times
service timestamps debug datetime msec
service timestamps log datetime msec
CORE-ROUTER#sh run | inc username
username admin privilege 15 secret 9 [HASH REMOVIDO]
CORE-ROUTER#sh run | sec vty
line vty 0 4
 exec-timeout 0 0
 activation-character 13
 login local
 transport input ssh
CORE-ROUTER#sh run | inc ntp master
CORE-ROUTER#sh run | inc ntp
CORE-ROUTER#

EDGE-ROUTER-1#sh clock
13:58:40.495 UTC Wed Sep 30 2026
EDGE-ROUTER-1#sh run | inc domain
ip domain name isp-lab.local
EDGE-ROUTER-1#sh run | inc buffer
logging buffered 64000
EDGE-ROUTER-1#sh run | inc times
service timestamps debug datetime msec
service timestamps log datetime msec
EDGE-ROUTER-1#sh run | inc username
username admin privilege 15 secret 9 [HASH REMOVIDO]
EDGE-ROUTER-1#sh run | sec vty
line vty 0 4
 exec-timeout 0 0
 activation-character 13
 login local
 transport input ssh
EDGE-ROUTER-1#sh run | sec interface
```

```
interface Loopback0
 ip address 2.2.2.2 255.255.255.255
interface Ethernet0/0
 description LINK-TO-CORE
 ip address 10.0.1.2 255.255.255.252
interface Ethernet0/1
 no ip address
 shutdown
interface Ethernet0/2
 no ip address
 shutdown
interface Ethernet0/3
 no ip address
 shutdown

EDGE-ROUTER-2#sh clock
13:59:38.609 UTC Wed Sep 30 2026
EDGE-ROUTER-2#sh run | inc domain
no ip domain lookup
ip domain name isp-lab.local
EDGE-ROUTER-2#sh run | inc buffere
logging buffered 64000
EDGE-ROUTER-2#sh run | inc times
service timestamps debug datetime msec
service timestamps log datetime msec
EDGE-ROUTER-2#sh run | inc username
username admin privilege 15 secret 9 [HASH REMOVIDO]
EDGE-ROUTER-2#sh run | sec vty
line vty 0 4
 exec-timeout 0 0
 activation-character 13
 login local
 transport input ssh
EDGE-ROUTER-2#sh run | sec interface
interface Loopback0
 ip address 3.3.3.3 255.255.255.255
interface Ethernet0/0
 description LINK-TO-CORE
 ip address 10.0.1.6 255.255.255.252
interface Ethernet0/1
 no ip address
 shutdown
interface Ethernet0/2
 no ip address
 shutdown
interface Ethernet0/3
 no ip address
 shutdown

EDGE-ROUTER-3#sh clock
14:00:24.904 UTC Wed Sep 30 2026
EDGE-ROUTER-3#sh run | inc domain
no ip domain lookup
ip domain name isp-lab.local
EDGE-ROUTER-3#sh run | inc buffered
logging buffered 64000
EDGE-ROUTER-3#sh run | inc times
service timestamps debug datetime msec
service timestamps log datetime msec
EDGE-ROUTER-3#sh run | inc username
username admin privilege 15 secret 9 [HASH REMOVIDO]
EDGE-ROUTER-3#sh run | sec vty
line vty 0 4
 exec-timeout 0 0
 activation-character 13
 login local
 transport input ssh
EDGE-ROUTER-3#sh run | sec interface
interface Loopback0
 ip address 4.4.4.4 255.255.255.255
```

```
interface Ethernet0/0
 description LINK-TO-CORE
 ip address 10.0.1.10 255.255.255.252
interface Ethernet0/1
 no ip address
 shutdown
interface Ethernet0/2
 no ip address
 shutdown
interface Ethernet0/3
 no ip address
 shutdown

--- Linux: auditoria automatizada ---
$ python 09_auditoria.py desvios
===== [D4-DESVIOS-R3] INICIO - AUDITORIA =====

core-router
  [FALHA] C07 core como servidor NTP

edge-router-1
  [FALHA] C01 no ip domain lookup

edge-router-2

edge-router-3

Resumo: 32 verificacoes em 4 roteadores | 2 FALHA(s) | 4 AVISO(s) | 4.3 s
===== [D4-DESVIOS-R3] FIM - AUDITORIA =====

--- Linux: gabarito e comparacao ---
$ python 09_auditoria.py --gabarito
===== [D4] GABARITO =====
  edge-router-1  regra C01  ->  ip domain lookup
  core-router    regra C07  ->  no ntp master

===== [D4] RODADA 3 - COMPARACAO COM O GABARITO (2 desvios injetados) =====
  Metodo            Tempo  Acertos  Falsos+  Nao encontrados
  manual           261.5s  2/2            0  -
  automatizado       4.3s  2/2            0  -
===== [D4] COMPARACAO - registrada em logs/resultados_D4_comparacao.csv =====

--- Linux: reversao ---
$ python 09_auditoria.py --reverter
===== [D4-DESVIOS-R3] DESVIOS REVERTIDOS =====
$ python 09_auditoria.py pos-reversao
Resumo: 32 verificacoes em 4 roteadores | 0 FALHA(s) | 4 AVISO(s) | 4.4 s
===== [D4-POS-REVERSAO] FIM - AUDITORIA =====
```

Resumo das três rodadas

```
$ python 09_auditoria.py --resumo
===== [D4] RESUMO DAS RODADAS =====
  Metodo        Rodadas      Deteccao  Falsos+     Media   Mediana
  manual              3    7/9     78%        0    346.1s    371.2s
  automatizado        3    9/9    100%        0      4.3s      4.3s
```

A.5. D5 – Contingência em falha de borda

Execução em 14/10/2026. Cenário de controle com a rede normal e cenário de falha com o roteador de borda 1 isolado pelo desligamento da interface Ethernet0/1 do *core*. O arquivo `evidencias/evidencias_D5.txt` contém o mesmo registro, com a tabela-resumo.

```
================================================================================
CENARIO 1 - CONTROLE (rede normal)
================================================================================
$ date
Wed Oct 14 10:07:53 -03 2026
$ python 02_backup.py
[OK]   core-router: backups/core-router_20261014-100820.cfg (146 linhas)
[OK]   edge-router-1: backups/edge-router-1_20261014-100820.cfg (139 linhas)
[OK]   edge-router-2: backups/edge-router-2_20261014-100820.cfg (140 linhas)
[OK]   edge-router-3: backups/edge-router-3_20261014-100820.cfg (139 linhas)

CORE-ROUTER#sh clock
13:07:09.852 UTC Wed Oct 14 2026
CORE-ROUTER#send log "===== [D5-CONTROLE] INICIO ====="

$ python 08_ativar_contingencia.py controle
===== [D5-CONTROLE] INICIO =====

>>> cliente-111: principal edge-router-1 acessivel (0.0 s) -> PRINCIPAL

=== cliente-111 EMPRESA-LAMBDA ===
  Configuracao gerada:
    interface Loopback111
     description CLIENTE:EMPRESA-LAMBDA | CIRCUITO:CORP-0111 | VEL:1G
     ip address 172.16.111.1 255.255.255.0
     no shutdown
    router bgp 65001
     network 172.16.111.0 mask 255.255.255.0
  [OK] BGP ok; ping 100% (3.7 s)

>>> cliente-112: principal edge-router-1 acessivel (0.0 s) -> PRINCIPAL

=== cliente-112 EMPRESA-MI ===
  Configuracao gerada:
    interface Loopback112
     description CLIENTE:EMPRESA-MI | CIRCUITO:CORP-0112 | VEL:500M
     ip address 172.16.112.1 255.255.255.0
     no shutdown
    router bgp 65001
     network 172.16.112.0 mask 255.255.255.0
  [OK] BGP ok; ping 100% (3.4 s)

>>> cliente-113: principal edge-router-1 acessivel (0.0 s) -> PRINCIPAL

=== cliente-113 EMPRESA-NI ===
  Configuracao gerada:
    interface Loopback113
     description CLIENTE:EMPRESA-NI | CIRCUITO:CORP-0113 | VEL:100M
     ip address 172.16.113.1 255.255.255.0
     no shutdown
    router bgp 65001
     network 172.16.113.0 mask 255.255.255.0
  [OK] BGP ok; ping 100% (3.4 s)

>>> cliente-114: principal edge-router-2 acessivel (0.0 s) -> PRINCIPAL

=== cliente-114 EMPRESA-XI ===
  Configuracao gerada:
    interface Loopback114
     description CLIENTE:EMPRESA-XI | CIRCUITO:CORP-0114 | VEL:200M
     ip address 172.16.114.1 255.255.255.0
```

```
     no shutdown
    router bgp 65001
     network 172.16.114.0 mask 255.255.255.0
  [OK] BGP ok; ping 100% (3.3 s)

>>> cliente-115: principal edge-router-2 acessivel (0.0 s) -> PRINCIPAL

=== cliente-115 EMPRESA-OMICRON ===
  Configuracao gerada:
    interface Loopback115
     description CLIENTE:EMPRESA-OMICRON | CIRCUITO:CORP-0115 | VEL:100M
     ip address 172.16.115.1 255.255.255.0
     no shutdown
    router bgp 65001
     network 172.16.115.0 mask 255.255.255.0
  [OK] BGP ok; ping 100% (3.7 s)

===== RESUMO D5-CONTROLE =====
Cliente Decisao       Roteador usado  Resultado     Total (s)
    111 PRINCIPAL     edge-router-1   OK                3.75
    112 PRINCIPAL     edge-router-1   OK                3.40
    113 PRINCIPAL     edge-router-1   OK                3.38
    114 PRINCIPAL     edge-router-2   OK                3.34
    115 PRINCIPAL     edge-router-2   OK                3.76
===== [D5-CONTROLE] FIM =====

CORE-ROUTER#show ip bgp | include 172.16.11
 *>i  172.16.111.0/24  2.2.2.2                  0    100      0 i
 *>i  172.16.112.0/24  2.2.2.2                  0    100      0 i
 *>i  172.16.113.0/24  2.2.2.2                  0    100      0 i
 *>i  172.16.114.0/24  3.3.3.3                  0    100      0 i
 *>i  172.16.115.0/24  3.3.3.3                  0    100      0 i
CORE-ROUTER#send log "===== [D5-CONTROLE] FIM ====="

$ python 08_ativar_contingencia.py --remover
cliente-111: removido
cliente-112: removido
cliente-113: removido
cliente-114: removido
cliente-115: removido

CORE-ROUTER#show ip bgp | include 172.16.11
CORE-ROUTER#

================================================================================
CENARIO 2 - FALHA DO EDGE-1
================================================================================
CORE-ROUTER#send log "===== [D5-FALHA] INICIO ====="
CORE-ROUTER#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
CORE-ROUTER(config)#interface Ethernet0/1
CORE-ROUTER(config-if)#shutdown
CORE-ROUTER(config-if)#end
CORE-ROUTER#show ip ospf neighbor

Neighbor ID     Pri   State           Dead Time   Address         Interface
4.4.4.4           1   FULL/DR         00:00:39    10.0.1.10       Ethernet0/3
3.3.3.3           1   FULL/DR         00:00:39    10.0.1.6        Ethernet0/2

$ echo "===== [D5-FALHA] INICIO ====="
===== [D5-FALHA] INICIO =====
$ ping -c 2 2.2.2.2
PING 2.2.2.2 (2.2.2.2) 56(84) bytes of data.
From 10.0.1.14 icmp_seq=1 Destination Host Unreachable
From 10.0.1.14 icmp_seq=2 Destination Host Unreachable

--- 2.2.2.2 ping statistics ---
2 packets transmitted, 0 received, +2 errors, 100% packet loss, time 1002ms

$ python 08_ativar_contingencia.py falha
```

```
===== [D5-FALHA] INICIO =====

>>> cliente-111: principal edge-router-1 INACESSIVEL (0.0 s) -> CONTINGENCIA

=== cliente-111 EMPRESA-LAMBDA ===
  Configuracao gerada:
    interface Loopback111
     description CLIENTE:EMPRESA-LAMBDA | CIRCUITO:CORP-0111 | VEL:1G
     ip address 172.16.111.1 255.255.255.0
     no shutdown
    router bgp 65001
     network 172.16.111.0 mask 255.255.255.0
  [OK] BGP ok; ping 100% (3.7 s)

>>> cliente-112: principal edge-router-1 INACESSIVEL (0.0 s) -> CONTINGENCIA

=== cliente-112 EMPRESA-MI ===
  Configuracao gerada:
    interface Loopback112
     description CLIENTE:EMPRESA-MI | CIRCUITO:CORP-0112 | VEL:500M
     ip address 172.16.112.1 255.255.255.0
     no shutdown
    router bgp 65001
     network 172.16.112.0 mask 255.255.255.0
  [OK] BGP ok; ping 100% (3.6 s)

>>> cliente-113: principal edge-router-1 INACESSIVEL (0.0 s) -> FALHA_SEGURA
  [FALHA SEGURA] edge-router-1 fora do ar e cliente sem acesso redundante: nenhuma configuracao
aplicada; abrir chamado

>>> cliente-114: principal edge-router-2 acessivel (0.0 s) -> PRINCIPAL

=== cliente-114 EMPRESA-XI ===
  Configuracao gerada:
    interface Loopback114
     description CLIENTE:EMPRESA-XI | CIRCUITO:CORP-0114 | VEL:200M
     ip address 172.16.114.1 255.255.255.0
     no shutdown
    router bgp 65001
     network 172.16.114.0 mask 255.255.255.0
  [OK] BGP ok; ping 100% (6.8 s)

>>> cliente-115: principal edge-router-2 acessivel (0.0 s) -> PRINCIPAL

=== cliente-115 EMPRESA-OMICRON ===
  Configuracao gerada:
    interface Loopback115
     description CLIENTE:EMPRESA-OMICRON | CIRCUITO:CORP-0115 | VEL:100M
     ip address 172.16.115.1 255.255.255.0
     no shutdown
    router bgp 65001
     network 172.16.115.0 mask 255.255.255.0
  [OK] BGP ok; ping 100% (6.7 s)

===== RESUMO D5-FALHA =====
Cliente Decisao       Roteador usado  Resultado     Total (s)
    111 CONTINGENCIA  edge-router-3   OK                 3.72
    112 CONTINGENCIA  edge-router-3   OK                 3.66
    113 FALHA_SEGURA  -               NAO_APLICADO       0.00
    114 PRINCIPAL     edge-router-2   OK                 6.87
    115 PRINCIPAL     edge-router-2   OK                 6.71
===== [D5-FALHA] FIM =====

CORE-ROUTER#show ip bgp | include 172.16.11
 *>i  172.16.111.0/24  4.4.4.4                  0    100      0 i
 *>i  172.16.112.0/24  4.4.4.4                  0    100      0 i
 *>i  172.16.114.0/24  3.3.3.3                  0    100      0 i
 *>i  172.16.115.0/24  3.3.3.3                  0    100      0 i
CORE-ROUTER#show ip bgp 172.16.113.0
% Network not in table
```

```
--- Restabelecimento do edge-1 ---
CORE-ROUTER#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
CORE-ROUTER(config)#interface Ethernet0/1
CORE-ROUTER(config-if)#no shutdown
CORE-ROUTER(config-if)#end
CORE-ROUTER#show ip bgp summary
BGP router identifier 1.1.1.1, local AS number 65001
BGP table version is 179, main routing table version 179
4 network entries using 992 bytes of memory
4 path entries using 544 bytes of memory
1/1 BGP path/bestpath attribute entries using 304 bytes of memory
0 BGP route-map cache entries using 0 bytes of memory
0 BGP filter-list cache entries using 0 bytes of memory
BGP using 1840 total bytes of memory
BGP activity 61/57 prefixes, 90/86 paths, scan interval 60 secs
10 networks peaked at 20:00:05 Sep 27 2026 UTC (2w2d ago)

Neighbor        V           AS MsgRcvd MsgSent   TblVer  InQ OutQ Up/Down  State/PfxRcd
2.2.2.2         4        65001       5       8      179    0    0 00:01:25        0
3.3.3.3         4        65001      34      41      179    0    0 00:23:40        2
4.4.4.4         4        65001      31      43      179    0    0 00:23:40        2

EDGE-ROUTER-1#show running-config | include Loopback11
EDGE-ROUTER-1#
(saida vazia: nenhuma configuracao do cliente 113 foi aplicada no edge-1)

CORE-ROUTER#send log "===== [D5-FALHA] FIM ====="
$ echo "===== [D5-FALHA] FIM ====="
===== [D5-FALHA] FIM =====

--- Limpeza ---
$ python 08_ativar_contingencia.py --remover
cliente-111: removido
cliente-112: removido
cliente-113: removido
cliente-114: removido
cliente-115: removido

CORE-ROUTER#show ip bgp | include 172.16.11
CORE-ROUTER#
```